

SI-C-004 — From Primitive Discovery to Basis Stress Testing

A Methodology for Testing Structural Intelligence Through Residuals, Counterexamples, and Cross-Domain Coverage

Sizhe Tan

with chatGPT (OpenAI)

2026-08-16

Repository: <https://github.com/sizhet/Structural-Intelligence-Compass>

Abstract

Structural Intelligence research has gradually accumulated a compact family of major and secondary computational structures.

Among the major structures, three broad compression mechanisms repeatedly appear:

1. **Organizational Compression;**
2. **Core-Preserved Generative Compression;**
3. **Calling-Path Compression.**

Around them, secondary structures such as Two-Way CCC, Universal Typing and Naming (UTN), Variable-Size Block Index, Function Tunnels, Runtime Invariants, Calling Graphs, triggering, switching, X-Y-M memory, feedback, and validation provide reusable computational support.

The emergence of a compact basis changes the appropriate research question.

The next step should not primarily be:

What new primitive can we invent?

A stronger question is:

What intelligent task cannot be naturally represented, organized, generated, routed, triggered, learned, executed, or validated using the current basis?

This paper proposes **Basis Stress Testing** as a methodological shift from primitive proliferation toward systematic falsification.

The central target is **Residual Intelligence**: the stable portion of an intelligent task that remains unexplained or computationally unnatural after serious attempts to construct the task using the current structural basis.

Residual Intelligence may indicate missing representation, identity,

organization, runtime control, validation, or a genuinely missing first-class primitive.

The objective is not to protect the current basis.

It is to discover where reality forces the basis to change.

A new primitive should ideally be discovered because a stable residual demands it, not because a theory looks incomplete without one.

1. A Research Program Changes When Its Primitive Rate Falls

Early in a new research program, major primitives may appear frequently.

A new problem reveals a new structural mechanism.

Another problem exposes another one.

The research landscape expands rapidly.

This is productive.

But a mature research program eventually faces a different possibility.

New problems may increasingly decompose into:

existing major structure

- **secondary operator**
- **runtime composition**

rather than requiring:

entirely new first-class primitive.

When this begins to happen repeatedly, the research question should change.

The relevant question is no longer:

What else can we add?

It becomes:

How hard can we attack what we already have before something genuinely new becomes necessary?

This is the transition from **Primitive Discovery** to **Basis Stress Testing**.

2. Why Continued Primitive Hunting Can Become Misleading

Primitive discovery is exciting.

It produces:

- new terminology;
- new diagrams;
- new algorithms;

- new papers;
- new architectures.

But once a plausible basis has formed, continued invention can create three problems.

2.1 Redundant Primitives

A newly named mechanism may be only:

- a special case;
- a composition;
- a parameterization;
- a secondary operator;
- a domain-specific implementation.

Promoting every useful mechanism to first-class status destroys explanatory economy.

2.2 Architectural Inflation

A theory can become difficult to use because every problem receives its own primitive.

The result may explain behavior descriptively while losing computational simplicity.

2.3 Avoidance of Falsification

Inventing new mechanisms is often easier than seriously testing old ones. Whenever a framework encounters difficulty, one can add another component.

This protects the framework from failure but weakens the research.

A stronger methodology deliberately exposes failure.

3. The Candidate Structural Basis

The current candidate basis begins with three major compression structures.

3.1 Organizational Compression

Large X-Space

→ **Differential Organization**

→ **Localized Structural Address**

Primary question:

Where does this state, object, or computation belong?

3.2 Core-Preserved Generative Compression

Large Generative Space

→ **Preserved Core**

- **Controlled Delta**

→ **Generated Result**

Primary question:

What must remain invariant while change is generated?

3.3 Calling-Path Compression

Large Experience Space

→ **Folded Computational Structure**

→ **Runtime Reconstruction**

Primary question:

What previously experienced structure can be compressed and reused?

4. Secondary Structural Mechanisms

The major basis is supported by smaller reusable mechanisms.

Important examples include:

- Two-Way CCC;
- UTN;
- Variable-Size Block Index;
- Differential Trees;
- Calling Graphs;
- Function Tunnels;
- Runtime Invariants;
- triggering;
- switching;
- X-Y-M memory;
- feedback;
- validation.

These mechanisms should not automatically be promoted into new first-class structures.

Their value may lie precisely in how broadly they support composition.

5. Why UTN Matters in Stress Testing

Universal Typing and Naming deserves special attention during Basis Stress Testing.

A system may appear to fail because its generation, organization, or runtime control is inadequate.

But the deeper problem may be identity.

Ask:

- Can the object be typed?
- Can it be named?

- Can the same object be referenced across transformations?
- Can two generated variants be recognized as belonging to the same structural identity?
- Can the system distinguish identity from implementation?

If not, the residual may be an **Identity Residual**, not evidence for a new top-level intelligence primitive.

UTN therefore helps prevent false primitive discovery.

6. Why Variable-Size Block Index Matters

A similar caution applies to structural span.

A task may appear impossible because the system uses the wrong unit size.

Fixed-size segmentation may fracture meaningful structures.

The problem may disappear if computation can address:

- variable-length blocks;
- nested regions;
- context-dependent units;
- hierarchical structures.

Therefore:

Before declaring a primitive residual, test whether the residual is actually an indexing or segmentation failure.

Variable-Size Block Index provides one mechanism for this test.

7. The Basis Stress Testing Principle

We can now state the central methodological rule.

Basis Stress Testing Principle

A candidate structural basis should be evaluated by attempting to construct increasingly diverse intelligent tasks from it and preserving any stable residuals that resist natural decomposition.

The phrase **natural decomposition** matters.

A framework should not be declared successful merely because anything can be simulated through enough arbitrary code.

The decomposition should remain:

- structurally interpretable;
- computationally economical;
- reusable;
- testable;
- consistent with the role assigned to the basis.

8. What Counts as a Natural Decomposition?

Suppose a task can technically be implemented using the current basis, but

only by adding:

- huge exceptions;
- opaque special cases;
- domain-specific hacks;
- duplicated logic;
- hidden universal modules.

That should not count as strong coverage.

A natural decomposition should explain why the task fits.

The major structure and operators should correspond to identifiable computational roles.

For example:

Organization

should actually reduce and localize the state space.

CG

should actually preserve a definable core.

Calling-Path Compression

should actually reuse folded experience.

Two-Way CCC

should actually expose preserve/subtract structure.

If the explanation becomes artificial, the residual may still be real.

9. Stress Testing Begins With Task Decomposition

For any candidate intelligent task, first ask what the task requires.

A general decomposition can include:

1. **Representation**
2. **Identity**
3. **Organization**
4. **Retrieval**
5. **Generation**
6. **Memory**
7. **Prediction**
8. **Decision**
9. **Calling**
10. **Triggering**
11. **Switching**
12. **Measurement**
13. **Feedback**
14. **Validation**
15. **Boundary Recognition**

This decomposition is not assumed to be final.

Its purpose is diagnostic.

It helps identify what kind of residual appears.

10. Residual Intelligence

The key concept is:

Residual Intelligence

Residual Intelligence is the portion of an intelligent task that cannot yet be naturally represented, organized, generated, routed, triggered, learned, executed, or validated using the current structural basis.

A residual is valuable because it marks a boundary.

It says:

Something here has not yet been explained economically.

That is exactly where new research should concentrate.

11. Residuals Should Be Preserved, Not Hidden

A common theoretical mistake is to explain away residuals.

If a framework predicts or constructs 90% of a task, the remaining 10% may be treated as implementation noise.

That can be dangerous.

The residual may contain the most important missing structure.

Therefore Basis Stress Testing adopts a different rule:

Do not smooth over the remainder. Characterize it.

Ask:

- Does it recur?
- Does it appear across domains?
- Does it resist alternative decompositions?
- Does it disappear after improving representation?
- Does it disappear after changing indexing?
- Does it disappear after adding runtime feedback?
- Does it require genuinely different computation?

12. A Taxonomy of Residual Intelligence

Residuals can arise at different layers.

Distinguishing them prevents premature primitive invention.

13. Representation Residual

A Representation Residual appears when important information cannot be expressed naturally in the current representational system.

Symptoms may include:

- unstable mapping;

- repeated information loss;
- inability to preserve critical distinctions;
- incompatible representations across modules.

Before adding a new primitive, ask:

Is the failure representational rather than computational?

14. Identity Residual

An Identity Residual appears when the system cannot reliably determine:

- what object is being referenced;
- whether two states correspond to the same entity;
- whether a generated variant preserves the original structural identity.

This is where UTN becomes especially relevant.

A missing identity layer can masquerade as a missing reasoning mechanism.

15. Segmentation Residual

A Segmentation Residual appears when the system uses the wrong computational unit.

A meaningful structure may span:

- multiple tokens;
- multiple nodes;
- variable-length sequences;
- nested blocks.

Fixed segmentation may prevent correct organization or preservation.

Variable-Size Block Index provides one candidate remedy.

16. Organizational Residual

An Organizational Residual appears when information exists but cannot be placed into a useful structure.

Symptoms include:

- poor localization;
- excessive global search;
- unstable routing;
- inability to reuse local experience.

This tests Organizational Compression directly.

17. Generative Residual

A Generative Residual appears when the system cannot produce needed variation while preserving the required core.

Symptoms include:

- semantic drift;

- broken invariants;
- interface violations;
- inability to distinguish core from editable structure.

This tests Core-Preserved Generation.

18. Experience Residual

An Experience Residual appears when previous computational experience cannot be efficiently reused.

Possible causes include:

- inadequate folding;
- inability to retrieve useful paths;
- catastrophic interference;
- inability to adapt compressed experience to new context.

This tests Calling-Path Compression.

19. Comparison Residual

A Comparison Residual appears when the system cannot identify meaningful preserved and differential structure.

This tests operators such as Two-Way CCC.

The issue may not require a new top-level primitive.

It may require a better comparison operator.

20. Runtime Residual

A Runtime Residual appears when the required capability exists somewhere in the system but cannot be activated at the correct time.

Symptoms include:

- wrong node selection;
- missed trigger;
- unstable runtime authority;
- unnecessary global activation.

This tests Calling Graphs, triggering, switching, and routing.

21. Memory Residual

A Memory Residual appears when useful runtime experience cannot influence future behavior appropriately.

X–Y–M memory provides a simple diagnostic structure:

X

→ **Y**

→ **M**

Ask:

- Was the state preserved?
- Was the decision recorded?
- Was the outcome measured?
- Was future behavior updated?

If not, the residual may be memory rather than intelligence.

22. Validation Residual

A Validation Residual appears when the system can produce a result but cannot reliably determine whether the result remains valid.

This can happen even when generation and decision are strong.

Validation deserves separate treatment.

A system that generates correctly most of the time but cannot identify failure conditions may still be unsafe.

23. Boundary Residual

A Boundary Residual appears when the system cannot recognize that it has crossed the region in which its own reasoning is validated.

This connects Basis Stress Testing to Boundary Intelligence.

The system may continue answering confidently when it should trigger:

- another perspective;
- another model;
- additional data;
- human review.

Boundary failure may therefore look like reasoning failure.

24. Primitive Residual

Only after the preceding possibilities have been examined should we consider:

Primitive Residual

A stable form of computation that appears to require a genuinely distinct first-class structure.

A Primitive Residual should ideally satisfy several conditions:

- it persists after representational improvement;
- it persists after better identity and indexing;
- it persists after runtime reorganization;
- it appears across multiple tasks;
- it is not merely a special case;
- introducing a new primitive materially simplifies the system.

This is a high bar.

It should be.

25. Criteria for Promoting a New Primitive

A candidate new primitive should ideally satisfy six tests.

Test 1 — Residual Necessity

Does it explain a stable residual?

Test 2 — Cross-Domain Recurrence

Does the same structure appear in more than one domain?

Test 3 — Non-Reducibility

Can it resist natural decomposition into existing structures?

Test 4 — Computational Economy

Does it simplify implementation rather than add another layer of description?

Test 5 — Reusability

Can the primitive serve multiple tasks?

Test 6 — Validation

Can its contribution be measured?

A mechanism that passes only one of these tests should probably remain an application technique or secondary operator.

26. Domain Stress Test — Scientific Discovery

Scientific discovery requires:

- representation;
- hypothesis generation;
- causal reasoning;
- experiment selection;
- validation;
- revision.

Candidate decomposition:

Organizational Compression

organizes known observations and hypotheses.

CG

generates candidate explanations while preserving constraints.

Calling-Path Compression

reuses learned scientific relationships.

Two-Way CCC

compares competing hypotheses.

UTN

preserves identity of variables, mechanisms, and hypotheses.

X–Y–M

records intervention and outcome.

The residual question is:

What part of genuine discovery still cannot be naturally constructed?

That remainder deserves close attention.

27. Domain Stress Test — Mathematics

Mathematical reasoning provides another challenge.

A system must handle:

- definitions;
- symbolic invariants;
- proof paths;
- abstraction;
- counterexamples.

Possible structural mapping:

UTN

for stable identity of definitions and objects.

Core Preservation

for invariant constraints.

Calling Paths

for reusable proof strategies.

Differential Organization

for theorem and structure organization.

Two-Way CCC

for comparing cases and counterexamples.

Residual question:

Does mathematical invention require a distinct first-class structure beyond organization, preservation, and reusable path compression?

This should be tested, not assumed.

28. Domain Stress Test — Robotics

Robotics adds embodiment and real-time control.

Requirements include:

- perception;
- state estimation;
- planning;
- action;
- feedback;
- switching.

Candidate mapping:

Differential Organization

for state-space localization.

Calling Graph

for potential actions.

Triggering / Switching

for runtime control.

X–Y–M

for action-effect memory.

Local Node Intelligence

for specialized control.

Residual question:

Does embodied intelligence expose a structural mechanism absent from disembodied tasks?

29. Domain Stress Test — Market Competition

Markets are especially useful because:

- state spaces are open;
- actors change strategy;
- exact repetition is rare;
- triggering can matter more than interpolation.

Candidate mapping:

X–Y–M

for market state, decision, outcome.

Two-Way CCC

for comparing successful and unsuccessful responses.

Multi-Perspective Function Tunnels

for alternative strategic views.

Triggering

for action thresholds.

Residual question:

Can strategic novelty be handled through structural extrapolation, or does it expose a new primitive?

30. Domain Stress Test — AI Coding

AI Coding provides a particularly strong test because several structures are simultaneously required.

UTN

preserves identity.

Variable-Size Block Index

provides flexible code-region addressing.

CG

preserves architecture and contracts while permitting implementation

variation.

Calling Graphs

represent dependencies.

Calling-Path Compression

provides learned coding experience.

Validation

tests runtime behavior.

Residual question:

What remains after generation, identity, structure, and validation are all explicitly accounted for?

31. Domain Stress Test — World Models

World Models test structural openness.

A capable World Model may require:

- objects;
- relations;
- transitions;
- dynamics;
- alternative perspectives;
- causal structure;
- uncertainty.

Candidate mapping:

UTN

for object identity.

Differential Organization

for world structure.

Calling Paths

for learned transitions.

CG

for candidate futures.

Boundary Intelligence

for validity limits.

Residual question:

Does world modeling require a fundamentally new structure, or mostly richer composition of existing ones?

32. Domain Stress Test — Biological Intelligence

Biological intelligence provides perhaps the most demanding comparison.

Brains exhibit:

- locality;

- specialization;
- memory;
- selective activation;
- feedback;
- multi-scale organization.

This does not prove that Structural Intelligence primitives map directly onto biology.

But biological systems can test whether:

broad intelligence plausibly emerges from repeated structural motifs rather than from a large catalog of unrelated primitives.

The objective is structural comparison, not biological reductionism.

33. Stress Test by Removal

Another useful method is not only to add tasks, but to remove components.

Ask:

What happens if this operator is absent?

For example:

Remove UTN.

Does identity become unstable?

Remove Two-Way CCC.

Does trigger discovery degrade?

Remove Local Node Intelligence.

Does global computation become inefficient?

Remove validation.

Does the system become unable to distinguish success from failure?

This helps determine which mechanisms are genuinely necessary.

34. Stress Test by Substitution

A mechanism may appear necessary only because no alternative has been tried.

Therefore test substitution.

Can:

UTN

be replaced by another identity mechanism?

Can:

Two-Way CCC

be replaced by another structural comparison method?

Can:

LLM

be replaced by another path-compression mechanism?

The purpose is not loyalty to names.
The purpose is identifying structural roles.

35. Stress Test by Scale

Some structures work at small scale but fail at large scale.

Test:

- number of nodes;
- number of objects;
- number of perspectives;
- number of runtime states;
- depth of Calling Graph;
- volume of X–Y–M memory.

A first-class structure should retain usefulness as the problem grows.

Scale failure may reveal:

- missing indexing;
- missing hierarchy;
- missing compression;
- missing locality.

36. Stress Test by Distribution Shift

A basis should not be evaluated only under familiar data.

Test:

- new objects;
- new categories;
- changed boundary conditions;
- unseen causal combinations;
- adversarial inputs;
- new tools;
- new actors.

This exposes whether the system is truly structural or merely memorizing one distribution.

37. Stress Test by Perspective Conflict

Run the same problem through different Function Tunnels.

For example:

- statistical;
- causal;
- LLM-based;
- simulation-based;
- rule-based.

Then ask:

- where do they agree?
- where do they disagree?
- which structure explains the difference?
- what confidence should be assigned?

This tests Cross-Perspective Recovery.

38. Stress Test by Runtime Switching

A system may appear intelligent under static conditions but fail when runtime authority must change.

Test:

Node A succeeds

then:

Boundary changes

then ask:

Can the system recognize the change and switch?

This evaluates:

- triggering;
- FTRI;
- policy routing;
- boundary intelligence.

39. Stress Test by Counterexample

Counterexamples should be actively designed.

A useful counterexample is one where:

Metric Similarity is High

but:

Structural Validity is Low

or:

Metric Similarity is Low

but:

Structural Validity is High.

These cases test whether the system relies on structure rather than superficial continuity.

40. Stress Test by Core Mutation

For CG, deliberately mutate:

- invariant core;
- optional region;
- identity;

- interface;
- behavior.

Then test whether the system can distinguish:

permitted delta

from:

core violation.

This directly measures Core-Preserved Generation quality.

41. Stress Test by Identity Drift

For UTN-supported systems, alter:

- names;
- implementations;
- locations;
- representations.

Ask:

Can the system still recognize the same structural object?

Then perform the reverse:

keep surface similarity while changing structural identity.

This tests whether UTN is merely labeling or actually supporting identity preservation.

42. Stress Test by Block Boundary

For Variable-Size Block Index, deliberately alter segmentation.

Test:

- too small;
- too large;
- nested;
- overlapping;
- dynamic boundaries.

If performance depends heavily on arbitrary segmentation, the structural representation may be weak.

43. Stress Test by Memory Conflict

X–Y–M memory may accumulate conflicting observations.

For one local state region:

X → Y1 → M1

X → Y2 → M2

The system must not simply average away meaningful differences.

This creates an opportunity for:

Two-Way CCC

→ **Preserve / Subtract**

→ **Candidate Trigger**

This stress test connects memory directly to generative trigger discovery.

44. From Multi-Valued X–Y–M to Trigger Synthesis

A particularly important future experiment is:

Single-Valued X–Y–M

→ **Multi-Valued X–Y–M**

→ **Outcome Grouping**

→ **Two-Way CCC**

→ **Preserved Structure**

- **Differential Structure**

→ **Trigger Hypothesis**

→ **Runtime Validation**

If successful, this would show that a secondary comparison operator can generate new runtime structure from accumulated experience.

This is exactly the kind of compositional power the Minimal Structural Basis Hypothesis predicts.

45. Stress Testing Should Measure More Than Accuracy

Accuracy alone is insufficient.

A basis may produce good answers for the wrong structural reasons.

Stress testing should also examine:

- interpretability of decomposition;
- runtime cost;
- structural locality;
- robustness;
- ability to detect boundaries;
- ease of validation;
- ability to incorporate new information;
- failure isolation.

A strong basis should improve architecture, not merely benchmark score.

46. Coverage Maps

One useful artifact from Basis Stress Testing is a **Coverage Map**.

For each domain, record:

- fully covered structures;
- partially covered structures;
- unresolved structures;
- discovered residuals.

Over time, this produces a structural map of where the basis works.
This is more informative than a binary success/failure claim.

47. Residual Maps

Residuals should also be organized.

A Residual Map can classify:

Representation

Identity

Organization

Generation

Experience

Runtime

Validation

Boundary

Primitive

If the same residual appears in multiple domains, its importance increases.

48. Cross-Domain Residual Clustering

Suppose:

robotics,

market competition,

and software runtime

all produce a similar unresolved switching problem.

That clustering may indicate a genuine missing structure.

Likewise, if scientific discovery and World Models both expose the same identity problem, UTN may need refinement.

Cross-domain clustering helps distinguish:

local engineering bug

from:

structural gap.

49. The Importance of Negative Results

A serious Basis Stress Testing program should publish negative results.

Examples:

The current basis failed to represent this type of recursive identity.

Two-Way CCC did not produce a stable trigger.

Core preservation failed under this type of semantic mutation.

These results are valuable.

They define the boundary.

Without them, a research program easily becomes self-confirming.

50. Basis Stress Testing as Collective Learning

This methodology is especially compatible with Collective Learning.

No single research group can test every domain.

The basis can be challenged independently by researchers working on:

- physics;
- robotics;
- finance;
- software engineering;
- biology;
- cognition;
- mathematics.

Each group can contribute a different stress surface.

This is stronger than merely asking others to adopt the framework.

The invitation becomes:

Attack it from where you know the territory best.

51. Why Invitational Research Matters

A theory can invite participation in two ways.

Weak invitation:

Please use our framework.

Strong invitation:

Please find where our framework fails.

The second creates more scientific value.

The Structural Intelligence Compass should prefer the stronger invitation.

52. Falsification Without Binary Rejection

Basis Stress Testing is not a simplistic pass/fail method.

A failed test does not necessarily mean:

the entire Structural Intelligence basis is wrong.

It may mean:

- one operator is missing;
- one representation is weak;
- one decomposition is wrong;
- one domain requires specialization.

The objective is structural diagnosis.

53. Revision Hierarchy

When a residual appears, revise in stages.

Level 1 — Implementation

Is the code wrong?

Level 2 — Parameterization

Is the mechanism poorly configured?

Level 3 — Representation

Is information encoded incorrectly?

Level 4 — Secondary Operator

Is a reusable operator missing?

Level 5 — Major Structure

Is one first-class structure inadequate?

Level 6 — Basis

Is a genuinely new first-class primitive required?

This hierarchy prevents premature theoretical expansion.

54. Occam's Razor for Structural Intelligence

A useful methodological rule is:

Add the smallest structural mechanism that explains the stable residual.

Do not add a new paradigm when a secondary operator is sufficient.

Do not add a secondary operator when better indexing is sufficient.

Do not add anything when the problem is implementation error.

This is Operator Economy applied to theory development.

55. Why a Small Basis Is Worth Defending Through Testing, Not Rhetoric

A small basis has important advantages.

It may provide:

- simpler Digital Brain architectures;
- easier validation;
- clearer runtime authority;
- stronger component reuse;
- more transparent failure analysis.

But these benefits make it tempting to become attached to minimality.

That would be a mistake.

Minimality should survive because reality permits it.

Not because researchers prefer it.

56. Evidence Has Priority Over Elegance

A three-part basis is elegant.

A 12-point Compass is symmetric.

Neither symmetry has scientific authority.

If a thirteenth Compass point is necessary, add it.

If a fourth major structure is necessary, add it.

If one of the current structures proves reducible, remove it.

The territory has authority over the Compass.

The same principle applies to the basis.

57. When Failure Strengthens the Framework

Paradoxically, finding a real residual can strengthen the research program.

Why?

Because it demonstrates that the framework is capable of exposing its own boundary.

A framework that can only confirm itself is weak.

A framework that can show:

Here is exactly where we stop explaining the task

is much more useful.

This is Boundary Intelligence applied to theory itself.

58. The Framework Should Have Boundary Intelligence Too

Structural Intelligence proposes Boundary Intelligence for intelligent systems.

The research program should obey the same principle.

It should explicitly represent:

- what the current basis explains;
- what remains uncertain;
- what is only hypothesized;
- what has failed validation.

In this sense, a research framework can practice the same epistemic discipline it recommends for AI.

59. Basis Stress Testing and Digital Brain Development

The methodology is especially important before large Digital Brain implementation.

Without stress testing, an architecture may accumulate components indefinitely.

With stress testing, each component must justify its structural role.

A Digital Brain can therefore evolve through:

Minimal Candidate Basis

→ **Prototype**

→ **Stress Test**

→ **Residual**

→ **Smallest Necessary Revision**

→ **Re-Test**

This is a much cleaner engineering path.

60. Basis Stress Testing and Hybrid Intelligence

Hybrid AI can easily become a component zoo.

LLM.

World Model.

Planner.

Tool system.

Simulator.

Rules.

Agents.

Human review.

Without structural analysis, each new failure may produce another module.

Basis Stress Testing asks instead:

Which structural role is actually missing?

Perhaps the system does not need another model.

Perhaps it needs:

- better routing;
- better identity;
- better trigger;
- better validation.

This distinction can prevent architectural overgrowth.

61. A Compact Stress-Test Template

For any new intelligent task, ask:

A. Identity

What are the structural objects?

Can UTN represent them?

B. Organization

Can Differential Organization localize the task?

C. Preservation

What invariants define correctness?

Can CG preserve them?

D. Experience

What prior paths can be reused?

Can Calling-Path Compression capture them?

E. Comparison

Can Two-Way CCC expose meaningful differences?

F. Runtime

Can Calling Graphs and triggering activate the right capability?

G. Memory

Can X–Y–M retain useful local outcomes?

H. Validation

Can the system measure whether it worked?

I. Boundary

Can it recognize when the basis is no longer sufficient?

J. Residual

What remains?

This template is intentionally simple.

Its purpose is to expose gaps.

62. Evidence Levels for a Candidate Residual

Not every failure deserves equal weight.

A residual can be classified as:

Level 0 — Anecdotal

One task failed.

Level 1 — Reproducible

The same task fails consistently.

Level 2 — Structural

A specific missing role can be identified.

Level 3 — Cross-Domain

The same residual appears across different domains.

Level 4 — Irreducible

Existing structures fail after serious alternative decomposition.

Level 5 — Primitive Candidate

A new compact structure explains the residual across domains.

Only Level 5 strongly supports promotion to a new first-class primitive.

63. Evidence Levels for Basis Strength

The opposite should also be measured.

Coverage Level 1

One domain.

Coverage Level 2

Several related domains.

Coverage Level 3

Multiple structurally different domains.

Coverage Level 4

Open-world tasks with distribution shift.

Coverage Level 5

Broad Digital Brain composition with stable validation and explicit boundaries.

This avoids vague claims that the basis is "general."

64. Basis Stress Testing Is Not Benchmark Chasing

Traditional benchmarks often ask:

How high is the score?

Basis Stress Testing asks:

Why did the system succeed or fail structurally?

A benchmark can be included.

But the structural decomposition is primary.

A lower-scoring system may teach more if it exposes a clean residual.

65. Stress Testing Is Also a Discovery Engine

The method is not merely defensive.

It can generate new ideas.

A residual may lead to:

- a new operator;
- a new representation;
- a new indexing method;
- a new trigger;
- a new confidence dimension;
- a new first-class primitive.

Thus stress testing replaces speculative invention with **evidence-driven invention**.

66. From Enumeration to Necessity

The research transition can now be stated clearly.

Old question:

What other intelligent mechanism can we enumerate?

New question:

What mechanism does a stable residual make necessary?

This is a major methodological improvement.

Necessity is stronger than novelty.

67. The Role of MET

Minimal Evolution Threshold remains relevant.

A candidate new primitive should represent a meaningful structural step.

If the observed improvement can be explained by:

- parameter tuning;
- scale;
- better data;
- composition;

then a new primitive may not have crossed MET.

Basis Stress Testing therefore helps determine whether an observed novelty represents:

quantitative improvement

or:

qualitative structural change.

68. Cross-Perspective Stress Testing

The Structural Intelligence Compass emphasizes perspective boundaries.

Basis testing should therefore include multiple perspectives.

For the same task:

Perspective A

→ decomposition A

Perspective B

→ decomposition B

Compare:

- common structures;
- conflicting residuals;
- shared failure modes.

If the same residual survives multiple perspectives, evidence for a missing primitive becomes stronger.

69. Residual Intelligence and Structural Extrapolation

Residuals may appear especially often near extrapolation boundaries.

A system can perform well in-distribution yet fail structurally when:

- actors change;
- boundary conditions change;
- causal relations shift;
- new object types appear.

This means Basis Stress Testing should deliberately operate beyond comfortable interpolation regions.

Otherwise important residuals remain hidden.

70. Residual Intelligence and Unknown

A mature system should be allowed to return:

Current basis insufficient.

This is not a failed test harness.

It is a meaningful result.

In runtime systems, that state can trigger:

- additional perspective;
- new data;
- escalation;
- experimentation.

In research, the same state can trigger:

- residual analysis;
- basis revision.

71. The Research Loop

The full methodology becomes:

Task

- **Structural Decomposition**
- **Candidate Basis Construction**
- **Implementation**
- **Validation**
- **Residual Detection**
- **Residual Classification**
- **Alternative Decomposition**
- **Cross-Domain Test**
- **Smallest Necessary Revision**
- **Re-Test**

This is an open loop.

72. What Success Looks Like

Two outcomes count as success.

Outcome A — Strong Basis Coverage

Increasingly diverse intelligent tasks can be naturally constructed from the same compact structural basis.

This supports:

- Operator Economy;
- compositional Digital Brain architecture;
- structural convergence.

Outcome B — Stable Residual

A recurring residual survives serious decomposition and cross-domain testing.

This exposes a missing structure.

Both outcomes advance knowledge.

73. What Failure Looks Like

The research program fails only if it does one of two things.

Failure A

It forces every observation into the current basis regardless of evidence.

Failure B

It invents a new primitive for every inconvenience without testing whether existing structures can explain it.

Both destroy explanatory discipline.

74. Collective Learning Protocol

The repository therefore invites outside researchers to contribute in a simple way:

1. choose a domain;
2. describe the intelligent task;
3. attempt structural decomposition;
4. map current basis components;
5. identify what fits naturally;
6. identify what does not;
7. preserve the residual;
8. propose the smallest structural revision necessary;
9. test again.

The objective is not to agree with Structural Intelligence.

The objective is to make the boundary visible.

75. Why This Is Invitational

A mature research framework should not say:

Come prove us right.

It should say:

Here is what we currently see.

Here is the candidate basis.

Here is how we think it can be tested.

Please show us where it stops working.

This is the spirit of the Structural Intelligence Compass.

76. Closing Perspective

The discovery of a compact structural basis is not the end of primitive research.

It changes the burden of proof.

A new primitive should no longer be accepted merely because it is interesting.

It should answer a stable structural need.

The path forward therefore becomes:

Use the basis.

Stress the basis.

Find the residual.

Classify the residual.

Revise only when necessary.

If no stable residual appears across increasingly broad domains, the evidence for a small structural basis grows.

If a stable residual appears, it provides a better starting point for the next discovery than speculation ever could.

That is the core methodological transition:

From Primitive Discovery

to:

Basis Stress Testing

and then, when reality requires it:

Residual-Driven Primitive Discovery

This sequence protects both openness and economy.

It allows the structural basis to remain small when possible.

It allows the basis to grow when necessary.

And it ensures that future growth is driven by the territory rather than by the desire to fill the map.

Final Methodological Principle

Do not ask first what new primitive can be invented. Ask what stable residual makes a new primitive necessary.

Structural Intelligence Compass

SI-C-004

Construct → Stress-Test → Preserve Residual → Revise → Re-Test

Try to break the basis. If it survives, learn from that. If it fails, learn even more.

